

Logical Operators

Logical Operators will allow us to combine multiple comparison operators. The logical operators we will learn about are:

- AND - &
- OR - |
- NOT - !

The best way to explain these is to see some examples.

In [1]:

```
# Imagine the variable x
x <- 10
```

Now we want to know if 10 is less than 20 AND greater than 5:

In [2]:

```
x < 20
```

Out[2]:

TRUE

In [3]:

```
x > 5
```

Out[3]:

TRUE

In [4]:

```
x < 20 & x > 5
```

Out[4]:

TRUE

We can also add parenthesis for readability and to make sure the order of comparisons is what we expect:

In [5]:

```
(x < 20) & (x > 5)
```

Out[5]:

TRUE

In [6]:

```
(x < 20) & (x > 5) & (x == 10)
```

Out[6]:

TRUE

We can basically think of this as a series of Logical Boolean values, TRUE & TRUE & TRUE. We return a single TRUE if they are all TRUE. Let's see an example of this not being the case:

In [7]:

```
x==2 & x > 1
```

Out[7]:

FALSE

Returned FALSE because while $x > 1$ is TRUE, we need BOTH to be TRUE, thus the AND statement &. What if we only want one of them to be true? That's when we use OR |. For example:

In [8]:

```
x==2 | x > 1
```

Out[8]:

TRUE

Here we only need one or the other to be true!

In [10]:

```
x==1 | x==12
```

Out[10]:

FALSE

NOT !

You can think about NOT as reversing any logical value in front of it, basically asking, "Is this NOT true?" For example:

In [13]:

```
(10==1)
```

Out[13]:

FALSE

In [14]:

```
!(10==1)
```

Out[14]:

TRUE

In [16]:

```
# We can stack them (pretty uncommon, but possible)
!!(10==1)
```

Out[16]:

FALSE

Use Case Example

Here's a quick example of a real use case for these operators. Imagine the following data frame:

In [32]:

```
df <- mtcars
```

In [33]:

```
df
```

Out[33]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.44	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.46	20.22	1	0	3	1
Duster 360	14.3	8	360	245	3.21	3.57	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.19	20	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.15	22.9	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.44	18.3	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.44	18.9	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.07	17.4	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.73	17.6	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.78	18	0	0	3	3
Cadillac Fleetwood	10.4	8	472	205	2.93	5.25	17.98	0	0	3	4
Lincoln Continental	10.4	8	460	215	3	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.2	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.9	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.7	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318	150	2.76	3.52	16.87	0	0	3	2
AMC Javelin	15.2	8	304	150	3.15	3.435	17.3	0	0	3	2
Camaro Z28	13.3	8	350	245	3.73	3.84	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79	66	4.08	1.935	18.9	1	1	4	1
Porsche 914-2	26	4	120.3	91	4.43	2.14	16.7	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Ford Pantera L	15.8	8	351	264	4.22	3.17	14.5	0	1	5	4
Ferrari Dino	19.7	6	145	175	3.62	2.77	15.5	0	1	5	6

Maserati Bora	15 mpg	8 cyl	301 disp	335 hp	3.54 drat	3.57 wt	14.6 qsec	0 vs	1 am	5 gear	8 carb
Volvo 142E	21.4	4	121	109	4.11	2.78	18.6	1	1	4	2

This shows some data for various car models (its built in to R). Let's grab models with at least 20 mpg:

In [39]:

```
df[df['mpg'] >= 20,] # Notice the use of indexing with the comma
# subset(df,mpg>=20) # Could also use subset
```

Out[39]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.32	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Merc 240D	24.4	4	146.7	62	3.69	3.19	20	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.15	22.9	1	0	4	2
Fiat 128	32.4	4	78.7	66	4.08	2.2	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.9	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.7	2.465	20.01	1	0	3	1
Fiat X1-9	27.3	4	79	66	4.08	1.935	18.9	1	1	4	1
Porsche 914-2	26	4	120.3	91	4.43	2.14	16.7	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Volvo 142E	21.4	4	121	109	4.11	2.78	18.6	1	1	4	2

Great! Now let's combine filters with logical operators! Let's grab rows with cars of at least 20mpg and over 100 hp.

In [41]:

```
df[(df['mpg'] >= 20) & (df['hp'] > 100),]
```

Out[41]:

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21	6	160	110	3.9	2.62	16.46	0	1	4	4
Mazda RX4 Wag	21	6	160	110	3.9	2.875	17.02	0	1	4	4
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
Volvo 142E	21.4	4	121	109	4.11	2.78	18.6	1	1	4	2

Hopefully you know see how useful these type of logical operators can be!

Logical Operators with Vectors

We have two options when use logical operators, a comparison of the entire vectors element by element, or just a comparison of the first elements in the vectors, to make sure the output is a single Logical, don't worry too much about this right now, we will cover it in more depth later on.

In [50]:

```
tf <- c(TRUE, FALSE)
tt <- c(TRUE, TRUE)
ft <- c(FALSE, TRUE)
```

In [51]:

```
tt & tf
```

Out[51]:

```
TRUE FALSE
```

In [55]:

```
tt | tf
```

Out[55]:

```
TRUE TRUE
```

To compare first elements use && or ||

In [56]:

```
ft && tt
```

Out[56]:

```
FALSE
```

In [57]:

```
tt && tf
```

Out[57]:

```
TRUE
```

In [58]:

```
tt || tf
```

Out[58]:

```
TRUE
```

In [59]:

```
tt || ft
```

Out[59]:

```
TRUE
```

Ok that's it for logical operators! This knowledge will help us when we use it for basic if and else statements!